

Off the shelf OCR and Deep Learning

Rohit Goswami · 2023-11-15 · python · ocr · tex

In general, [detexify](#) ([here](#)) and [my ScanTools workflow](#) is great. However, sometimes more can be done.

```
micromamba create -n textocr
micromamba activate textocr
micromamba install torchvision -c pytorch
pip install pix2tex[gui]
pip install python-doctr
pip install nougat-ocr
```

The `TeX` tool ([LaTeX OCR](#)) works great even via the terminal. The `doctr` library is a bit more finicky, but can be a decent way to extract plain text when regular OCR tools fail (e.g. `ocrmypdf`).

```
import argparse
from pathlib import Path
from doctr.io import DocumentFile
from doctr.models import ocr_predictor

def process_pdf(pdf_path):
    model = ocr_predictor(
        det_arch="db_resnet50", reco_arch="crnn_vgg16_bn", pretrained=True
    )
    doc = DocumentFile.from_pdf(pdf_path)
    result = model(doc)
    return result.render()

def save_to_text(input_path, output_text):
    output_file = Path(input_path).with_suffix(".txt")
    output_file.write_text(output_text, encoding="utf-8")

def main():
    parser = argparse.ArgumentParser(description="OCR processing of a PDF file")
    parser.add_argument("pdf_file", help="Path to the PDF file to be processed")
    args = parser.parse_args()
    ocr_result = process_pdf(args.pdf_file)
    save_to_text(args.pdf_file, ocr_result)

if __name__ == "__main__":
    main()
```

Which is alright, called via `python -c doctr_runner.py blah.pdf`.

Or [Meta's nougat](#), which is slower but generally better formatted:

```
nougat blah.pdf -o output_dir
```

As of `15-11-2023`, both these options have [a known warning](#) about a memory leak.